USAGE OF MIXED INTEGER LINEAR PROGRAMMING IN CRYPTANALYSIS OF BLOCK CIPHERS

Halil İbrahim Kaplan

Istanbul Technical University
Informatics Institute

2025

Table of Contents

Table of Contents

Block Ciphers

Block ciphers are main elements in symmetric cryptography, designed to securely encrypt and decrypt fixed-size blocks of data by transforming plaintext into ciphertext using a symmetric key. k
Figure ekle!!!!

Block Cipher types

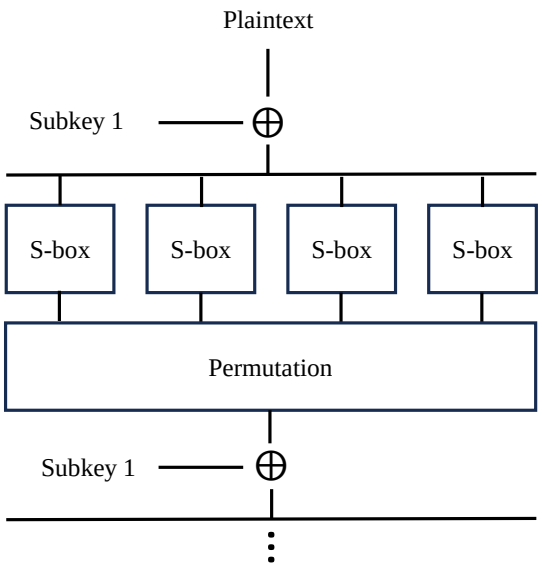


Figure 1: SPN Cipher

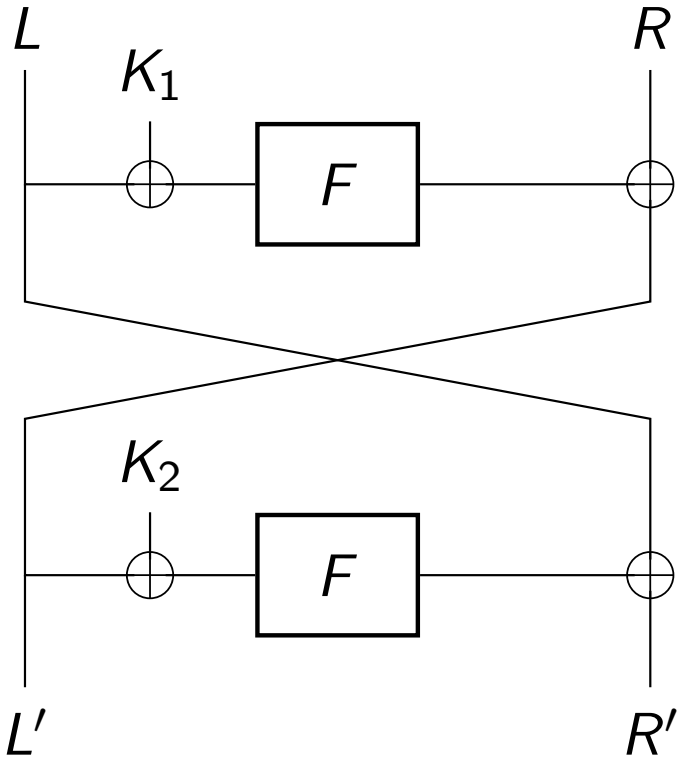


Figure 2: Fiestel Structure

Cryptanalysis

The main purpose of cryptanalysis is to prove that an algorithm is secure against known attacks in the open literature or find better complexities for those attacks and contribute to the security of the algorithm. In cryptanalysis, breaking an algorithm means obtaining the key with a lower complexity than the algorithm claims.

AES-128 Security Against Cryptanalytic Attacks

Attack Type	**Best Known Result**	**Complexity (Time / Data)**	**Full 10-Round AES-128**	**Reference**
Brute Force	Exhaustive key search	2^{128} time, negligible data	Infeasible	[5]
Linear Cryptanalysis	Works up to 7 rounds	$\approx 2^{126.1}$ time, $\approx 2^{126.1}$ data	No distinguisher	[9, 5]
Differential Cryptanalysis	Works up to 7 rounds	$\approx 2^{126.1}$ time, $\approx 2^{126.1}$ data	No distinguisher	[1, 5]
Impossible Differential	Attacks up to 6 rounds	$\approx 2^{110}$ time, $\approx 2^{110}$ data	Not effective	[2]
Integral (Square) Attack	Effective up to 6–8 rounds	$\approx 2^{124}$ time, full codebook data	No attack	[4, 7]
Related-Key Attacks	Attacks up to 9 rounds	$2^{99.5}$ time, 2^{32} related keys	No single-key break	[2]
Algebraic Attacks	Solve AES equations via Gröbner basis	$> 2^{128}$ time	Not effective	[3]

Table of Contents

Linear Programming

The standard form of a linear programming (LP) problem is:

Maximize (or Minimize):

$$c_1x_1 + c_2x_2 + \cdots + c_nx_n$$

Subject to:

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &\leq b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &\leq b_2 \\ &\vdots \\ a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n &\leq b_m \\ x_1, x_2, \ldots, x_n &\geq 0 \end{aligned}$$

Here, $x_1, x_2, \ldots, x_n$ are the **decision variables**, $c_1, c_2, \ldots, c_n$ are the **coefficients of the objective function**, a_{ij} are the **constraint coefficients**, and $b_1, b_2, \ldots, b_m$ are the **constraint bounds**.

Mixed-Integer Linear Programming

Mixed-Integer Linear Programming (MILP) extends LP by requiring some variables to take integer values:

Maximize (or Minimize):

$$c_1x_1 + c_2x_2 + \cdots + c_nx_n$$

Subject to:

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &\leq b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &\leq b_2 \\ &\vdots \\ a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n &\leq b_m \\ x_1, x_2, \ldots, x_p &\in \mathbb{Z} \\ x_{p+1}, x_{p+2}, \ldots, x_n &\geq 0 \end{aligned}$$

In this formulation, the first p decision variables $(x_1, x_2, \ldots, x_p)$ must be **integers**, while the remaining $n - p$ variables $(x_{p+1}, \ldots, x_n)$ are **real-valued**.

MILP example

A company produces two types of products, **Product A** and **Product B**. The goal is to maximize profit while considering production capacity, resource availability, and a minimum production requirement. The constraints involve available labor hours, machine hours, and minimum production levels.

MILP example

Variables:

- x be the number of units produced of **Product A**.
- y be the number of units produced of **Product B**.

Both x and y are integer variables.

Objective Function:

Maximize the total profit: Maximize $Z = 40x + 30y$ where:

- 40 is the profit per unit of **Product A**,
- 30 is the profit per unit of **Product B**.

MILP example

Constraints

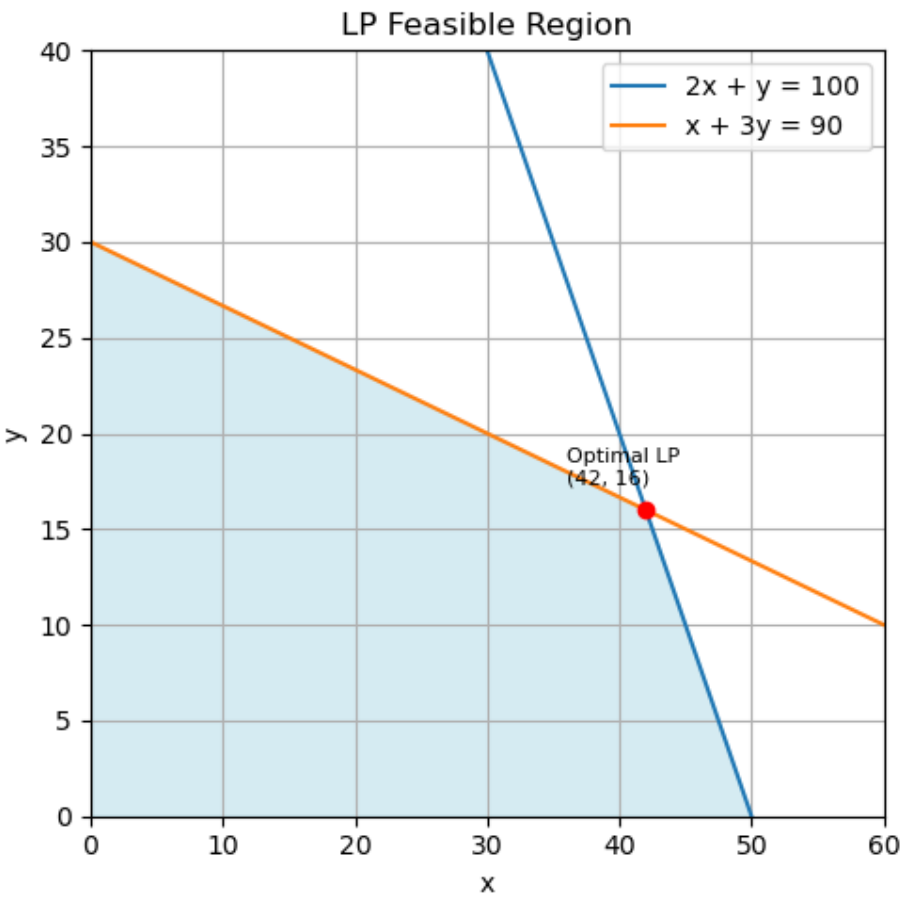
1. **Labor Constraint**: Each unit of **Product A** requires 2 hours of labor, and each unit of **Product B** requires 1 hour of labor. The total available labor hours are 100. $2x + y \leq 100$
2. **Machine Hours Constraint**: Each unit of **Product A** requires 1 hour of machine time, and each unit of **Product B** requires 3 hours of machine time. The total available machine hours are 90. $x + 3y \leq 90$
3. **Minimum Production Constraint**: At least 20 units of **Product A** must be produced to meet market demand. $x \geq 20$
4. **Non-Negativity Constraint**: The number of units produced cannot be negative. $x \geq 0, \quad y \geq 0$

MILP example

We will use Sagemath and GLPK solver for modeling Toy example. After running the below code, we found that Maximum profit 2160 can be achieved by producing 42 A product and 16 B product.

```
p.<x,y> = MixedIntegerLinearProgram(maximization=True,
    solver="GLPK")
p.set_objective(40*x[0] + 30*y[0])
p.add_constraint(2*x[0] + y[0] == 100)
p.add_constraint(x[0] + 3*y[0] == 90)
p.add_constraint(x[0] >= 20)
p.add_constraint(y[0] >= 0)
print(p.solve())
print(p.get_values(x, y))
```

MILP example



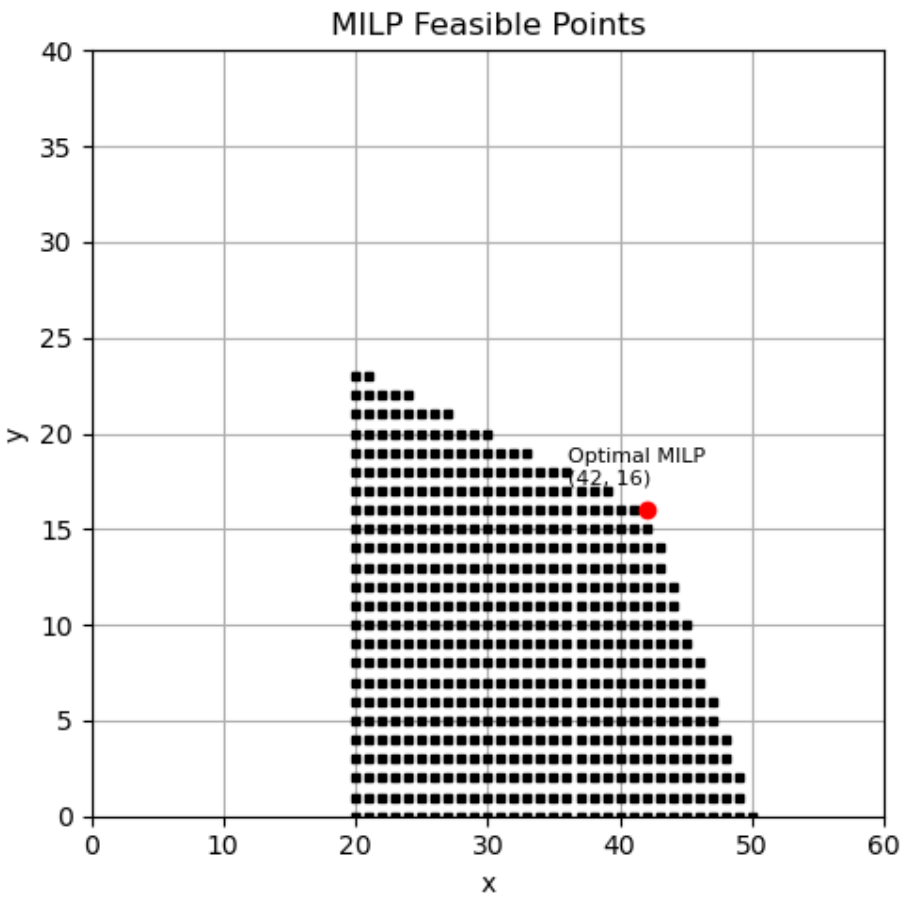


Table of Contents

Differential Cryptanalysis

Differential cryptalysis is chosen plaintext attack founded by Bilham and Shamir against Data Encryption Standart (DES) Once the effectiveness of the attack against DES was seen, it became standard to apply differential attacks to demonstrate the security of all block cipher algorithms.

Differential Cryptanalysis

Definition

Assume that P and P' two plaintexts then difference ΔP is defined as $\Delta P = P \oplus P'$

Differential Cryptanalysis leverages the high probability of specific plaintext differences ΔX and the differences in the cipher's final round ΔY.

Differential Cryptanalysis

In an ideal cipher:

- For a given ΔX, the probability that a specific ΔY occurs is $1/2^n$ (where n is the number of bits in X).
- Differential Cryptanalysis aims to find instances where, for a given ΔX, the probability of a specific ΔY occurring is significantly higher or lower than $1/2^n$.

Differential Cryptanalysis

Definition

Diff characteristic is sequence of input and output differences to the rounds so that the output difference from one round corresponds to the input difference for the next round.

In order to construct differential characteristic for the cipher we need to find differential for each round with very high or low occurrences. To find thees characteristics we need to examine the nonlinear parts of the cipher like s-boxes. To examine the s-boxes, we use difference distribution tables (DDT)

Differential Cryptanalysis

fig/difchar.png

Differential Cryptanalysis

Definition

DDT is a matrix that tabulates the number of occurrences of specific output differences (ΔY) for each possible input difference (ΔX) after a single round of a cipher.

Algorithm 1: Pseudocode for DDT

Initialize an empty DDT table

for $0 \leq dx < 2^n - 1$ **do**

  for $0 \leq x < 2^n - 1$ **do**

    $x1 = dx \oplus x$

    $y = sbox(x)$

    $y1 = sbox(x1)$

    $dy = y \oplus y1$

    $DDT[dx][dy] += 1$

Differential Cryptanalysis

$\Delta X/\Delta Y$	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	16	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	4	4	4	4	0	0	0	0
2	0	4	0	4	0	4	4	0	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0	2	2	2	2	2	2	2	2
4	0	0	4	0	0	0	2	2	0	0	0	4	2	2	0	0
5	0	0	4	0	0	0	2	2	0	0	4	0	2	2	0	0
6	0	2	0	2	2	0	0	2	2	0	2	0	0	2	2	0
7	0	2	0	2	2	0	0	2	0	2	0	2	2	0	0	2
8	0	0	0	0	4	4	0	0	0	0	0	0	2	2	2	2
9	0	0	0	0	4	4	0	0	0	0	0	0	2	2	2	2
A	0	0	0	0	0	4	4	0	2	2	2	2	0	0	0	0
B	0	4	0	4	0	0	0	0	0	0	0	0	2	2	2	2
C	0	0	4	0	0	0	2	2	4	0	0	0	0	0	2	2
D	0	0	4	0	0	0	2	2	0	4	0	0	0	0	2	2
E	0	2	0	2	2	0	0	2	0	2	0	2	0	2	2	0
F	0	2	0	2	2	0	0	2	2	0	2	0	2	0	0	2

Table 1: Differential Distribution Table (DDT) of PRINCE algorithm

Differential Cryptanalysis

1. **Selection of Differentials:**
   - Identify differentials $(\Delta X, \Delta Y)$ with non-random propagation probabilities by examining the cipher's S-boxes using the Differential Distribution Table (DDT).
   - Select differentials with the highest probabilities for the analysis.
2. **Collection of Pairs:**
   - Obtain a large number of plaintext pairs (P, P') such that $P \oplus P' = \Delta X$.
3. **Cipher Execution:**
   - Encrypt the selected plaintext pairs through the cipher.

Differential Cryptanalysis

4. **Analysis of Output Differences:**
   - Analyze the differences in the output ciphertext pairs.
   - Check if the observed differences match the expected ΔY.
5. **Key Recovery:**
   - Use the observed differentials and the associated probabilities to guess parts of the encryption key, focusing particularly on the subkeys used in the last few rounds.

Table of Contents

ITUbee Block Cipher

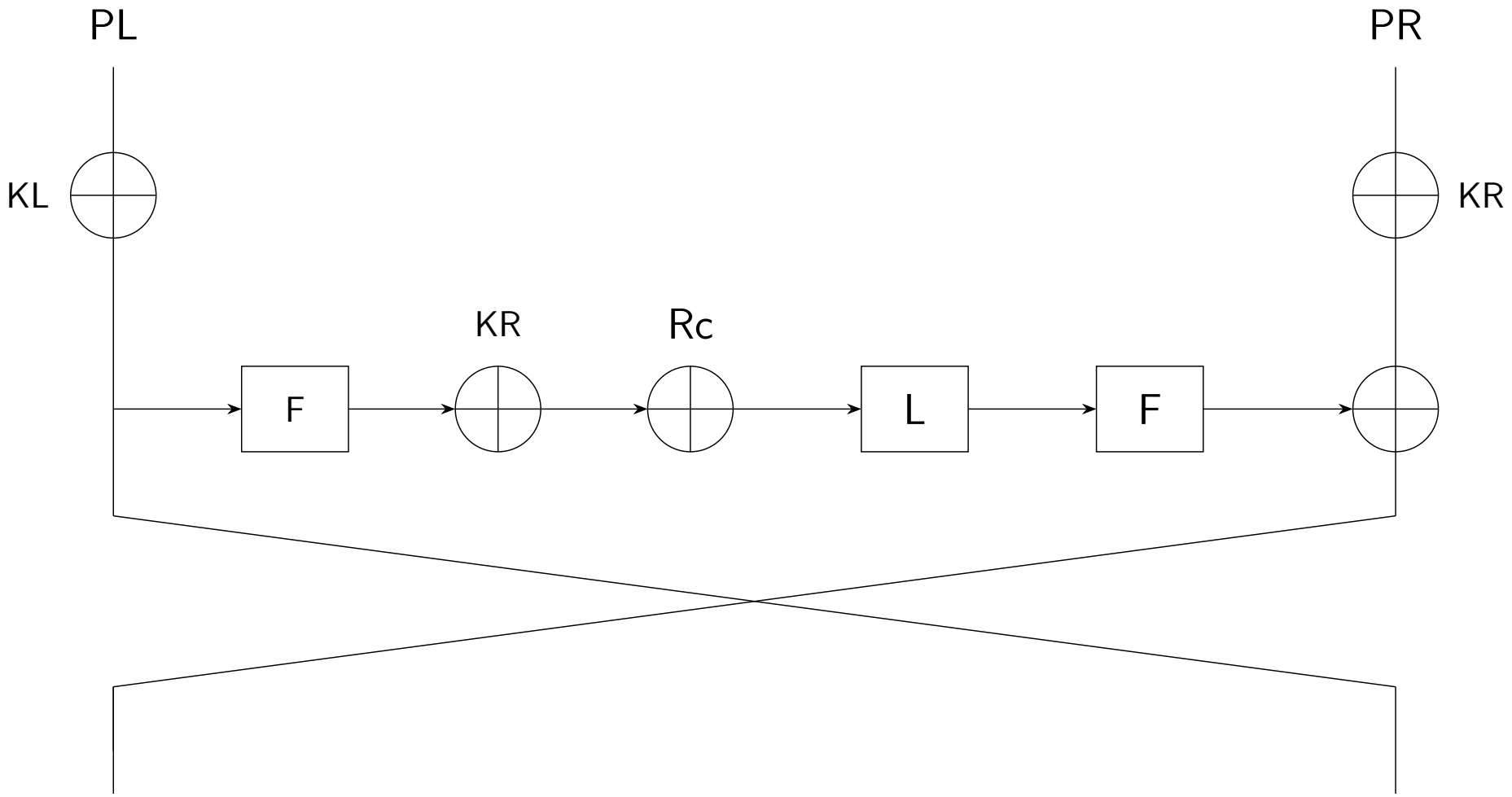


Figure 4: ITUbee round structure

ITUbee Block Cipher

Algorithm 2: Pseudo code for ITUbee encryption

$X_1 \leftarrow P_L \oplus K_L$
$X_0 \leftarrow P_R \oplus K_R$
for $i = 1$ *to* 20 **do**
  if $i \in \{1, 3, \ldots, 19\}$ **then**
    $RK \leftarrow K_R$
  else
    $RK \leftarrow K_L$
  $X_{i+1} \leftarrow X_{i-1} \oplus F(L(RK \oplus RC_i \oplus F(X_i)))$
  `// Round constant is added to the rightmost 16 bits`
$C_L \leftarrow X_{20} \oplus K_R$
$C_R \leftarrow X_{21} \oplus K_L$

L Function

The L function is computed as,
$L(a \parallel b \parallel c \parallel d \parallel e) = (e \oplus a \oplus b) \parallel (a \oplus b \oplus c) \parallel (b \oplus c \oplus d) \parallel (c \oplus d \oplus e) \parallel (d \oplus e \oplus a)$
This function can also be represented as matrix multiplication using rotational matrix:

$$\begin{pmatrix} 1 & 1 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 & 1 \end{pmatrix} \begin{pmatrix} a \\ b \\ c \\ d \\ e \end{pmatrix} = \begin{pmatrix} e \oplus a \oplus b \\ a \oplus b \oplus c \\ b \oplus c \oplus d \\ c \oplus d \oplus e \\ d \oplus e \oplus a \end{pmatrix}$$

F function

The F function is computed as,

$$F(X) = S(L(S(X)))$$

S is the S-box used in AES [6]. S is the only non-linear structure in the ITUbee algorithm. 40 bit input of S is splitted into 8 bits then enters the s-box.

$$S(a \parallel b \parallel c \parallel d \parallel e) = s[a] \parallel s[b] \parallel s[c] \parallel s[d] \parallel s[e]$$

Key Schedule

ITUbee has no key schedule. The algorithm splits the 80 bit key into two 40 bit parts K_L and K_R and uses the right part of the key in odd rounds and the left part of the keys in even rounds.

Table of Contents

Branch Number

If we consider a transformation $\mathcal{T}$ applied to an n-element vector, the branch number is given by:

$$B = \min_{\mathbf{x} \neq \mathbf{0}} \{\mathrm{wt}(\mathbf{x}) + \mathrm{wt}(\mathcal{T}(\mathbf{x}))\},$$

where $\mathrm{wt}(\mathbf{x})$ denotes the Hamming weight of the vector $\mathbf{x}$, i.e., the number of non-zero elements in $\mathbf{x}$.

A higher branch number indicates better diffusion.

Branch Number Example

For AES [6], the MDS matrix used in the MixColumns step has a branch number of 5. This means that minimum number of total active byte of input and output of the MDS is 5 Table **??** shows possible active byte patterns of the AES MDS matrix.

Input Active Bytes	**Output Active Bytes**
1	4
2	3 / 4
3	2 / 3 / 4
4	1 / 2 / 3 / 4

Table 2: Possible active byte patterns of the AES MDS matrix

Milp Modelling of XOR Operation

We will use modelling defined in [10]. Lets x_{in1} and x_{in2} be input and x_{out} be output difference of xor operation. Branch number of xor operation is 2. We use a dummy variable d to add the branch number to the equation system. The dummy variable is equal to 0 if x_{in1}, x_{in2} and x_{out} are equal to 0, otherwise it is equal to 1. XOR operation can be expressed with the following equations:

$$
\begin{gathered}
x_{in1} + x_{in2} + x_{out} \geq 2d \\
d \geq x_{in1} \\
d \geq x_{in2} \\
d \geq x_{out}
\end{gathered}
$$

Milp Modelling of Linear Transformation

We will also use modelling defined in [10]. Lets $x_{in1}, x_{in2}, ..., x_{inM}$, be input differences and $x_{out1}, x_{out2}, ..., x_{outM}$, be output difference of linear transformation. We will use dummy variable d like in XOR modelling and we have branch number B of Linear transformation. Then we can define linear transformation with following equations.

Milp Modelling of Linear Transformation

$$x_{in1} + x_{in2} + x_{inM} + ... + x_{out1} + x_{out2} + ... + x_{outM} \geq B * d$$
$$d \geq x_{in1}$$
$$d \geq x_{in2}$$
$$...$$
$$d \geq x_{inM}$$
$$d \geq x_{out1}$$
$$d \geq x_{out2}$$
$$...$$
$$d \geq x_{outM}$$

In block ciphers, any nonlinear structure can be considered as a linear transformation. For example, xor operation or matrix multiplication are linear transformations. When the branch number value is given as 2 in the equation above, the equations given for XOR operation are obtained.

MILP Representation of S-boxes

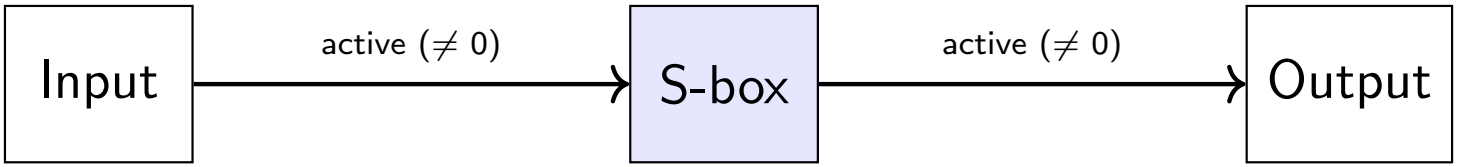
In MILP-based differential cryptanalysis, our goal is to minimize the number of active S-boxes to find the best differential trails. Hence, S-boxes are represented abstractly without modeling their internal logic.

Key Observation:

- If the input to an S-box is active (non-zero difference), the output is also active.
- If the input is passive (zero difference), the output is passive.

This behavior allows us to model the S-box using a binary activity variable without considering its internal structure.

MILP Representation of S-boxes



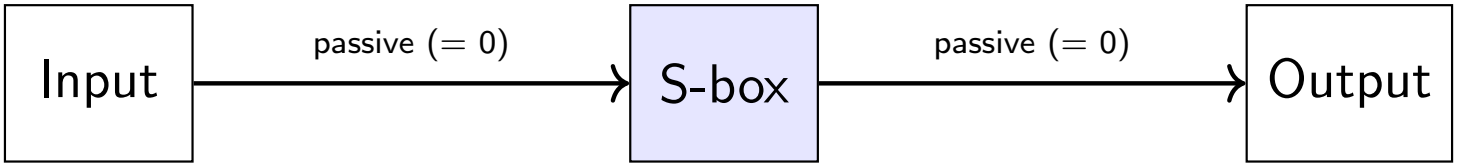


Table of Contents

H-representation of a Polyhedron

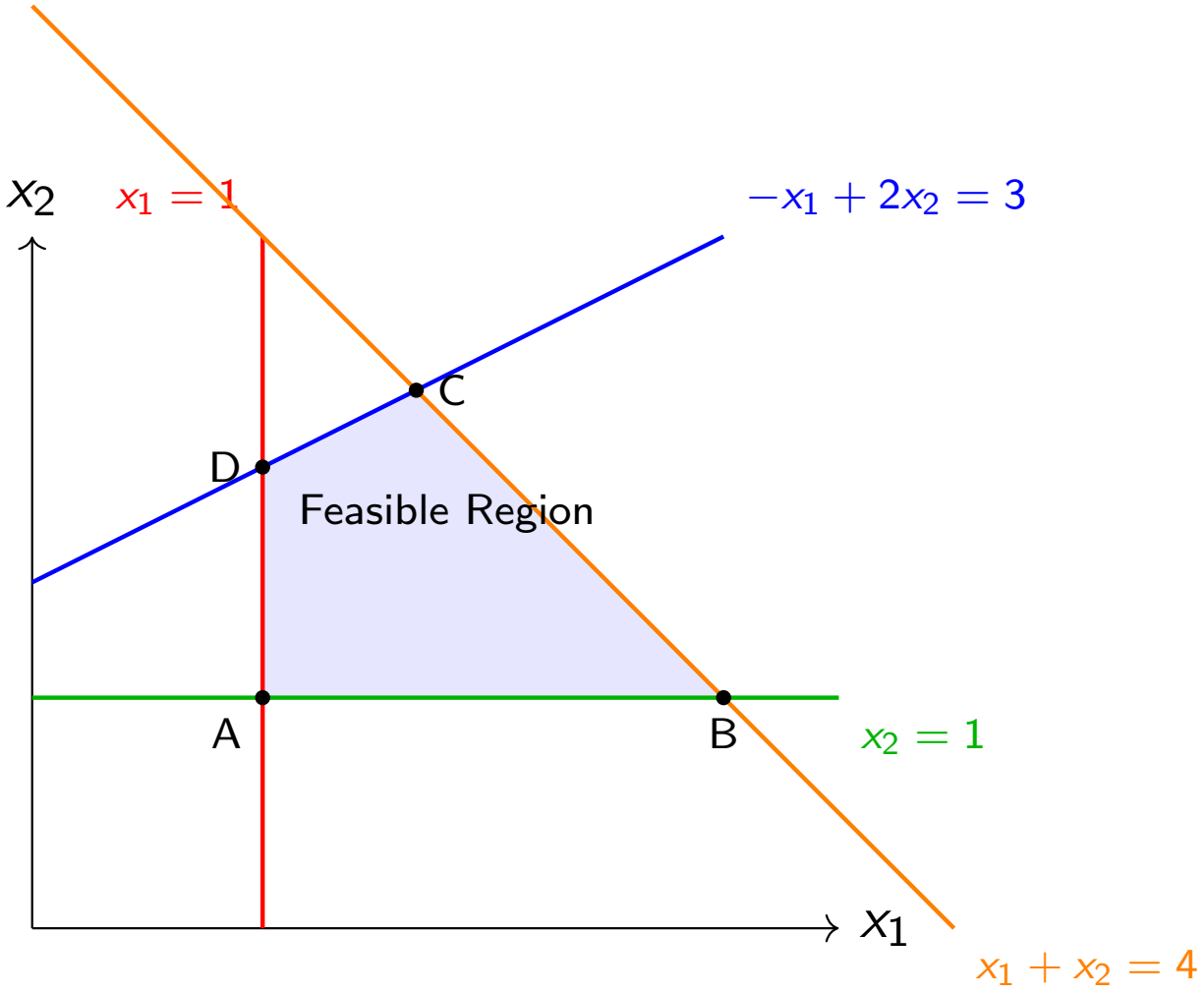
The **H-representation** (half-space representation) defines a polyhedron as the intersection of a finite number of half-spaces. Each half-space is defined by a linear inequality:

$$\mathbf{a}\mathbf{x} \leq b$$

- $\mathbf{a} \in \mathbb{R}^n$: vector of coefficients.
- $\mathbf{x} \in \mathbb{R}^n$: vector of variables.
- $b \in \mathbb{R}$: scalar constant.

H-representation of a Polyhedron

Example: A 2D polygon defined by the intersection of 4 half-spaces:



H-representation

This representation is integral to the MILP modeling of block ciphers as it encapsulates the constraints from the cipher's operations, including the behavior of S-boxes, linear layers, and key additions. In the context of cryptanalysis, using the H-representation allows complex relationships between variables to be expressed linearly, enabling MILP solvers to explore the solution.

H-representation example

Consider linear function L which takes 6 bit input, 6 bit output and computed as:

$$L(a, b, c) = (a \oplus b, a \oplus c, b \oplus c) \text{ where a,b.c are 2 bit values}$$

H-representation of this function can be found in 2 steps with using Sagemath polyhedra library. [11]

H-representation example

1. We scan the outputs for all possible inputs and give a value of 0 if the output value is 0 and 1 if it is different from zero. Possible input and output values of the L function can be seen in Table **??**.
2. These points are given as input to the sage polyhedra library and the inequalities given in table **??** are obtained.

H-representation example

Input Differences	Output Differences
0 0 0	0 0 0
0 0 1	0 1 1
0 1 0	1 0 1
0 1 1	1 1 0
0 1 1	1 1 1
1 0 0	1 1 0
1 0 1	1 0 1
1 0 1	1 1 1
1 1 0	0 1 1
...	...
1 1 1	1 0 1
1 1 1	1 1 0
1 1 1	1 1 1

Table 3: Input and Output Differences of L function

H-representation example

No.	Inequality
1	$(0, 0, -1, 0, 0, 0)x + 1 \geq 0$
2	$(0, -1, 0, 0, 0, 0)x + 1 \geq 0$
3	$(-1, 0, 0, 0, 0, 0)x + 1 \geq 0$
4	$(0, 0, 0, -1, 0, 0)x + 1 \geq 0$
5	$(0, 0, 0, 0, -1, 0)x + 1 \geq 0$
6	$(0, 0, 0, 0, 0, -1)x + 1 \geq 0$
7	$(0, -1, 1, 0, 0, 1)x + 0 \geq 0$
8	$(0, 0, 0, -1, 1, 1)x + 0 \geq 0$
...	...
18	$(-1, 1, 0, 1, 0, 0)x + 0 \geq 0$
19	$(1, -1, -1, 1, 1, -1)x + 1 \geq 0$
20	$(1, -1, 0, 1, 0, 0)x + 0 \geq 0$
21	$(-1, 1, -1, 1, -1, 1)x + 1 \geq 0$
22	$(-1, -1, 1, -1, 1, 1)x + 1 \geq 0$

Table 4: H-representation of L function

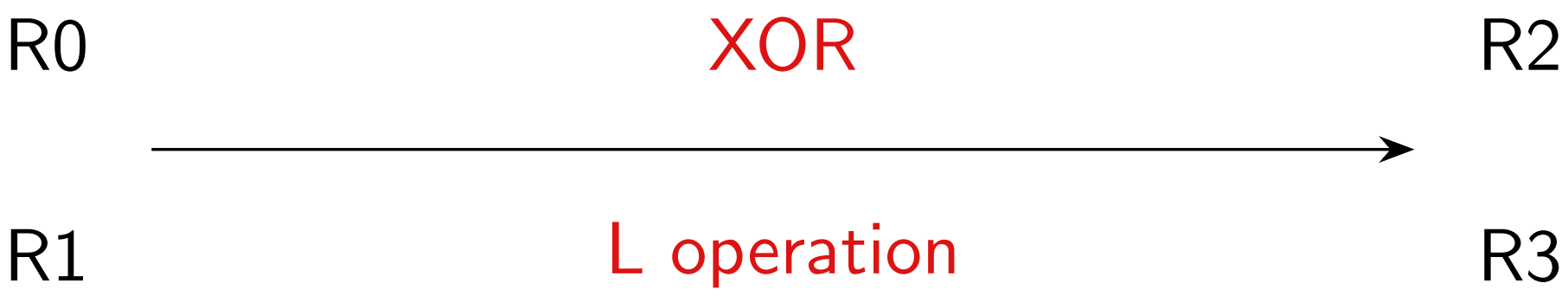
Table of Contents

Differential Cryptanalysis Model Using MILP

The ITUbee algorithm has a Feistel structure, so we represent the input as two variables, $R0$ and $R1$, each modeled as binary variables indicating activity status.
In the MILP model, after each operation in the cipher, we introduce new binary variables whenever a byte's activity status can change—i.e., an active byte may become passive or vice versa.
However, the S-box operation does not alter the activity status of a byte: if the input byte is active, the output byte remains active, and if inactive, it remains inactive. Therefore, no new binary variables are introduced after S-box layers.

Differential cryptanalysis model with MILP

L operations are changes the active bytes so after them we define new variable. XOR operation changes active byte if it XOR two binary variables. In differential cryptanalysis we do not give active values to the keys therefore key xor's does not change the active byte. Skech of the 1 round modelling can be seen in the Figure **??**.



Differential cryptanalysis model with MILP

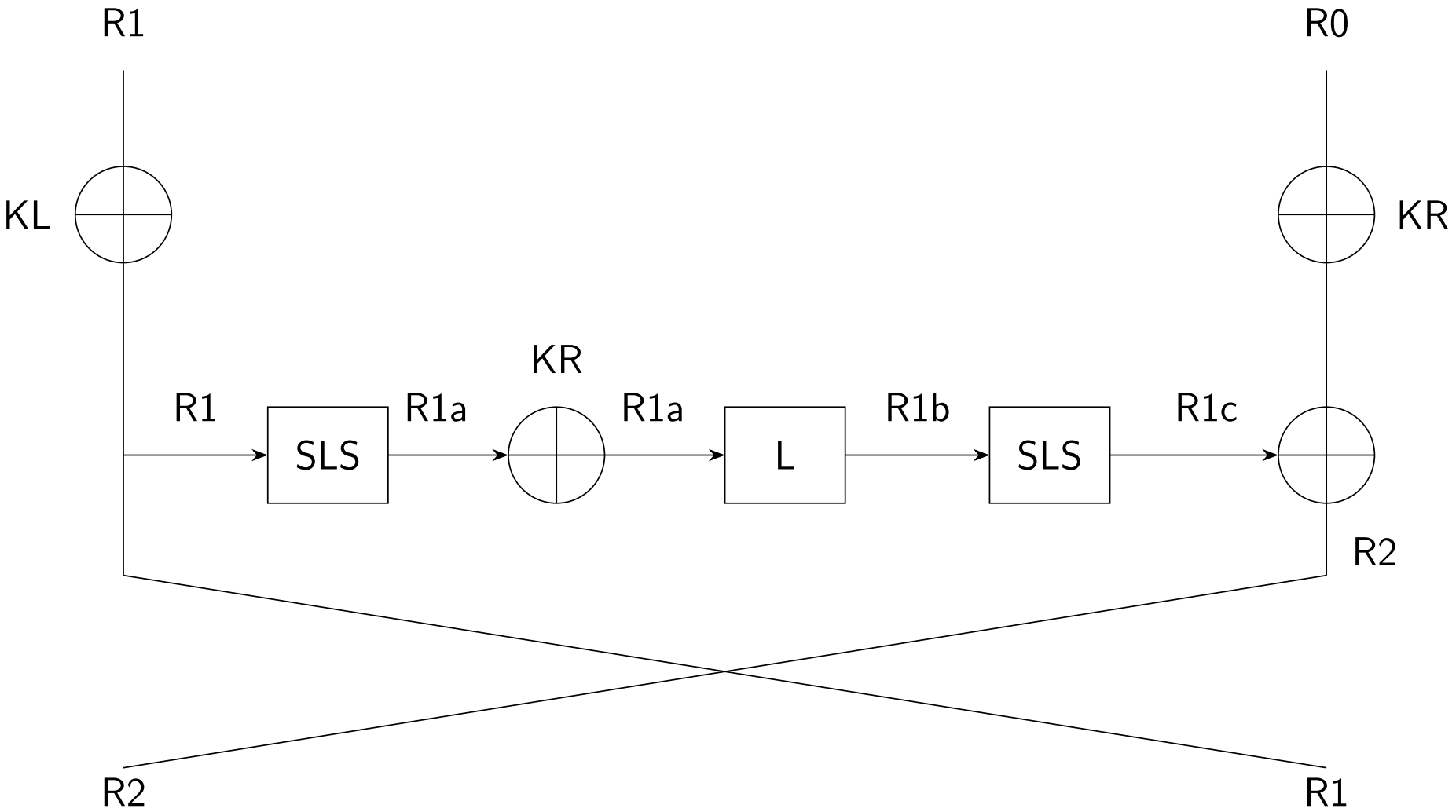


Figure 5: ITUbee MILP differential cryptanalysis sketch

Differential Cryptanalysis Model with MILP

- In MILP-based differential cryptanalysis, our goal is to find the best differential characteristic by minimizing the number of active S-boxes.
- We model binary variables before and after each S-box and minimize their sum to find the characteristic with the lowest activity.
- The ITUbee block cipher uses the AES S-box [6], which has a maximum differential probability of 2^{-6}.

Model Output (3 Rounds)

- The best characteristic found activates 16 S-boxes over 3 rounds.
- Therefore, the maximum differential probability is:

$$(2^{-6})^{16} = 2^{-96}$$

Differential Cryptanalysis Model with MILP

- A differential characteristic with probability 2^{-96} is too low to be useful for a practical attack.
- This result implies that 3 rounds of the ITUbee cipher are secure against differential cryptanalysis.
- Our findings are consistent with those reported in [8].

Characteristic Table

The best differential characteristic found by the MILP model is shown in Table **??**.

Differential cryptanalysis model with MILP

Stage	Binary Index	Characteristic
1. round input	R1	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 1.0}
	R0	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
1. round middle values	R1a	{0: 1.0, 1: 0.0, 2: 0.0, 3: 1.0, 4: 1.0}
	R1b	{0: 0.0, 1: 1.0, 2: 1.0, 3: 0.0, 4: 1.0}
	R1c	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 1.0}
2. round input	R2	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 1.0}
	R1	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 1.0}
2. round middle values	R2a	{0: 1.0, 1: 0.0, 2: 0.0, 3: 1.0, 4: 1.0}
	R2b	{0: 0.0, 1: 1.0, 2: 1.0, 3: 0.0, 4: 1.0}
	R2c	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 1.0}
3. round input	R3	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
	R2	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 1.0}
3. round middle values	R3a	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
	R3b	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
	R3c	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
4. round input	R4	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 1.0}
	R3	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}

Table 5: Best 3 round differential characteristic of ITUbee algorithm

Table of Contents

References

[1] Eli Biham and Adi Shamir. "Differential Cryptanalysis of DES-like Cryptosystems". In: *Journal of Cryptology* 4.1 (1991), pp. 3–72.

[2] Alex Biryukov and Dmitry Khovratovich. "Related-key Cryptanalysis of the Full AES-192 and AES-256". In: *ASIACRYPT*. Springer, 2009.

[3] Nicolas Courtois and Josef Pieprzyk. "Cryptanalysis of Block Ciphers with Overdefined Systems of Equations". In: *ASIACRYPT*. Springer, 2002.

[4] Joan Daemen, Lars R. Knudsen, and Vincent Rijmen. "The Block Cipher Square". In: *FSE*. Springer, 1997.

[5] Joan Daemen and Vincent Rijmen. *The Design of Rijndael: AES - The Advanced Encryption Standard*. Springer, 2002.

[6] Morris J Dworkin et al. "Advanced encryption standard (AES)". In: (2001).

[7] Niels Ferguson, Richard Schroeppel, and Doug Whiting. *A simple algebraic representation of Rijndael*. Tech. rep. Citeseer, 2000.